PROGREE

DEVOPS INTERNSHIP REPORT

Remote Internship Program

5th July 2026 – 4th August 2026

Intern Name	**Fouenang Miguel Bruce**
Student ID	**B2/724**
Position	**Remote Devops Intern**
Organisation	**Progree**
Report Date	**4 August 2026**
Project Repository	**NetOps-hub**

TABLE OF CONTENTS

1.Executive Summary

This report documents the completion of a four-task DevOps internship program at Progree. Over the course of one month (5th July 2026 to 4th August 2026), I designed, built, and deployed **NetOps-Hub** — a comprehensive Network Operations and Automation Platform. This full-stack application demonstrates practical application of containerization, continuous integration/continuous deployment (CI/CD), Infrastructure as Code (IaC), and Kubernetes orchestration, while leveraging my background in Linux administration, and full-stack development.

Project	Backend	Frontend	Devops	Networking
NetOps_hub Network Operation Center and Automation Platform	FastAPI,python3.12,SqlAlchemy 2.0	HTML5,CSS3,Bootstrap 5.3,Vanilla Javascript	Docker,Docker-Compose,Github Actions,K	Netmiko,ICMP ping,SSH automation,TCP/IP

2. Professional Linkedin Announcement

2.1 Objective

Share the official internship acceptance on LinkedIn to build professional visibility and network within the DevOps community.

2.2 Execution

- Posted a high-resolution copy of the official Internship Offer Letter on LinkedIn.
- Crafted a compelling caption detailing learning goals in DevOps, Docker, CI/CD, and cloud infrastructure.
- Tagged Progree's official LinkedIn page
- Used required hashtags: **#devops #docker #cicd #progree**

2.3 Outcome

- Increased professional network visibility
- Established personal brand in the DevOps space

- Connected with peers and mentors in the industry

For this tasks this my Post like and Screenshot of the link of up to today

[Task1 Internship Link]

Fouenang Miguel Bruce • You
--Software & Network Engineer | Python/Flask Developer | Linux Admin | ...
3w •

Very Excited to Begin My DevOps Internship Journey!

I'm thrilled to share that I have officially joined Progree as a Remote DevOps Intern.

As a 4th-year Network & Telecommunications Engineering student at the National Advanced School of Engineering (ENSP Douala), this opportunity represents an important milestone in my journey toward becoming a DevOps Engineer.

Throughout this internship, I look forward to:

- 🐧 Strengthening my Linux system administration skills
- 🐳 Working with Docker and containerized applications
- ☁️ Exploring cloud technologies and DevOps best practices
- ⚙️ Learning more about CI/CD pipelines and automation
- 🤝 Collaborating on real-world projects and gaining valuable industry experience

I'm grateful to the Progree team for this opportunity and excited to learn from experienced professionals while contributing to meaningful projects.

This is only the beginning, and I'm looking forward to sharing my learning journey and the projects I'll be working on over the coming weeks.

Thank you to everyone who has supported and encouraged me along the way!

#DevOps #Internship #Linux #Docker #CloudComputing #Automation #CICD #Networking #Python #EngineeringStudent #ContinuousLearning #CareerGrowth

Progree
Your Career Our Mission

INTERNSHIP OFFER LETTER

25 June 2026
Name: Fouenang Miguel Bruce STUDENT ID: B2/724
Dear Intern, Congratulations!
We are pleased to offer you the position of Remote Internship in
DevOps
This letter confirms your selection for our **Remote Internship Program**, which will be conducted from **5th July 2026 to 4th August 2026**.
This internship has been designed to provide you with valuable learning opportunities

Progree
Your Career Our Mission

INTERNSHIP OFFER LETTER

25 June 2026
Name: Fouenang Miguel Bruce STUDENT ID: B2/724
Dear Intern, Congratulations!
We are pleased to offer you the position of Remote Internship in
DevOps
This letter confirms your selection for our **Remote Internship Program**, which will be conducted from **5th July 2026 to 4th August 2026**.
This internship has been designed to provide you with valuable learning opportunities and practical industry experience. Throughout the program, you will receive guidance, training, and mentorship to help you enhance your technical skills and gain a deeper understanding of development through hands-on projects and assignments.
As an intern, you are expected to:
- **Complete all assigned tasks and projects responsibly.**
- **Follow the instructions and guidelines provided by the Progree Team.**
- **Maintain professionalism and commitment throughout the internship period.**
- **Participate actively in learning activities and training sessions.**

We are excited to have you join our team and look forward to supporting your professional growth. We believe this internship will help you build the skills, knowledge, and experience needed for future career opportunities.
We wish you a rewarding, enjoyable, and successful internship experience with Progree.

Sincerely,

Umar Mushtaq
Founder & CEO

APPROVED Progree APPROVED

Phone: +92 314 1816693
Email: progree.edu@gmail.com
Address: Rawalpindi , Pakistan

Agassem Sanda Florentin and 17 others 2 comments

Like Comment Repost Send

595 impressions View analytics

Add a comment... GIF

[Project-repo]

3. APPLICATION CONTAINERIZATION & ASSET OPTIMIZATION

3.1 Objective

Build a modular, reliable application container environment package with minimal image footprint, secure environment variable handling, and functional port routing.

3.2 Project: NetOps-Hub

NetOps-Hub is a full-stack Network Operations and Automation Platform built with **FastAPI**, **PostgreSQL**, and modern web technologies. It provides network engineers and DevOps professionals with a centralized dashboard for device management, network automation, and configuration backup.

3.2.1 Core Application Features

Features	Description
Device management	Complete CRUD operation with search and filter capacity
ICMP ping Testing	Latency and packet loss reporting for network reachability
SSH connectivity	Remote command execution(e.g show version)
Configuration Backup/Restore	Automated backup using Netmiko
Activity Logging	Comprehensive audit trail of all user/system actions
Responsive Dashboard	Real-time statistics with interactive charts

3.2.2 Technology Stack

Layer	Technology
Backend	FastAPI,python3.12,SQLAlchemy 2.0
Database	PortgreSQL with pooling and migrations
Frontend	HTML5,CSS3,Bootstrap 5.3,Vanilla Javascript
Network Automation	Netmiko,ICMP ping,SSH automation,TCP/IP

3.2.3 Multi-Stage Dockerfile Architecture

The Dockerfile implements a two-stage build process to optimize image size and security:

Stage	Purpose	Content
BUILDER	Compile and Validate	Python 3.12 + build deps, all requirements, compiled static assets
PRODUCTION	Runtime Only	Python 3.12 Slim, runtime dependencies only, non-root user

3.2.4 Image Optimization Results

Metrics	Before	After	Improvement
Image Size	577 MB	234 MB	343 MB reduction (59.4%)
Build Tools	Included	Removed	Faster pulls, less storage
Security	Root user	Non-root user	Reduced attack surface

3.2.5 Security Best Practices Applied

- Non-root execution: Application runs as unprivileged user
- Minimal base image: Python slim variant removes unnecessary packages
- Optimized layer caching: Dependency files copied before source code
- No hardcoded secrets: All configuration via environment variables
- Docker Compose: Multi-service orchestration with PostgreSQL service

3.2.6 Docker Compose Configuration

The application includes a docker-compose.yml for local development and testing, orchestrating the web service, PostgreSQL database service, health checks, and restart policies.

3.3 Outcome

- Production-ready containerized application with 59.4% image size reduction
- Multi-service orchestration via Docker Compose
- Full test coverage with Pytest and service mocking
- Demonstrate networking skills through integrated network automation tools

Project Repo

4. MULTI-STAGE AUTOMATED CI/CD DEPLOYMENT PIPELINE

4.1 Objective

Script an automated testing and integration workflow that triggers on remote pushes, executes linting and testing, builds Docker images, and logs deployment metrics.

4.2 CI/CD Pipeline Architecture (GitHub Actions)

The pipeline is implemented using GitHub Actions with three distinct stages:

Stage	Trigger	Action
TEST & VALIDATE	**Push to main or PR**	**Checkout, Setup Python 3.12, Install deps, Start PostgreSQL, Run Pytest, Code quality checks**
BUILD & PUSH	**Tests pass**	**Setup Docker Buildx, Login Docker Hub, Build multi-stage image, Tag with SHA + latest, Push to Docker Hub**
VERIFY & NOTIFY	**Image pushed**	**Validate pushed image, GitHub UI feedback, Branch protection enforcement**

4.3 Pipeline Configuration Highlights

Component	Tool	Purpose
CI Platform	GitHub Actions	Free, integrated, industry-standard
Testing	Pytest with service mocking	Unit and integration testing
Database	PostgreSQL service	Realistic testing environment
Container Build	Docker Buildx	Multi-platform, cached builds
Registry	Docker Hub	Public image distribution
Secrets	GitHub Secrets	Secure credential management
Branch Protection	GitHub Rules	Only main branch triggers production builds

4.4 Automation Coverage

Trigger	Action
Code Push	Automatically triggers CI/CD pipeline
Pull Request	Runs tests before merge approval
Successful Tests	Automatic Docker image build and push
Docker Hub	Image tagged with commit SHA and latest
GitHub UI	Immediate feedback on code quality

4.5 Secrets Management

Secure handling of sensitive credentials using GitHub Secrets:

- DOCKER_USERNAME: Docker Hub ID
- DOCKER_PASSWORD: Docker Hub Access Token
- Database credentials via environment variables
- No secrets committed to version control

4.6 Outcome

- Fully automated pipeline with zero manual intervention
- Every code push triggers complete validation chain
- Docker images versioned with Git SHA for traceability
- Branch protection ensures only tested code reaches production

Project-repo

5. AUTOMATED INFRASTRUCTURE AS CODE & ORCHESTRATION ARCHITECTURE

5.1 Objective

Provision and orchestrate an elastic, highly available microservices cluster environment using declarative IaC tools and Kubernetes.

5.2 Kubernetes Architecture

The Kubernetes manifests define a complete deployment stack within the netops-hub namespace:

Component	Resource	Purpose
Namespace	netops-hub	Logical isolation of all application resources
Deployment	netops-hub-deployment	Application pods with 2 replicas, resource limits, health probes
Service	netops-hub-service	ClusterIP for internal load balancing (Port 80 -> 8000)
PVC	netops-hub-logs-pvc	Persistent storage for configuration backup retention (1Gi)

5.3 Deployment Configuration

The deployment manifest includes production-grade specifications:

Spec	Value	Purpose
Replicas	2	High availability with failover capability
Resource Requests	128Mi memory, 100m CPU	Guaranteed baseline resources
Resource Limits	256Mi memory, 200m CPU	Prevent resource exhaustion
Security Context	runAsNonRoot: true	Prevent privilege escalation

Liveness Probe	HTTP GET /health, 30s interval	Restart unhealthy pods automatically
Readiness Probe	HTTP GET /health, 10s interval	Only route traffic to ready pods

5.4 Status: Partially Completed

Component	Status	Notes
Namespace	Completed	netops-hub namespace created
Deployment	Completed	Full deployment manifest with security contexts
Service	Completed	ClusterIP service for internal routing
PVC	Completed	Persistent volume for log retention
Ingress	Ready	Manifest prepared, pending cluster deployment
HPA	Ready	Configuration defined, pending metrics server

5.5 Next Steps for Full Completion

1. 1. Apply manifests to live Kubernetes cluster (EKS/GKE/Minikube)
2. 2. Configure Ingress Controller (NGINX) for external access
3. 3. Enable Horizontal Pod Autoscaler based on CPU/memory metrics
4. 4. Integrate ConfigMaps/Secrets for environment-specific configuration
5. 5. Set up Persistent Volumes for configuration backup retention

6. SKILLS & TECHNOLOGIES APPLIED

6.1 Core DevOps Skills

Skill	Application
Containerization	Multi-stage Docker builds, Docker Compose orchestration
CI/CD	GitHub Actions, automated testing, Docker registry
IaC	Kubernetes manifests for declarative deployment
Orchestration	Kubernetes deployments, services, PVCs
Monitoring	Health checks, activity logging, audit trails
Security	Non-root containers, secrets management, least privilege

6.2 Technologies Used

Category	Technologies
Languages	Python 3.12, JavaScript, Bash, YAML
Framework	FastAPI, SQLAlchemy 2.0
Database	PostgreSQL with connection pooling
Frontend	HTML5, CSS3, Bootstrap 5.3,JS
Containers	Docker, Docker Compose
Orchestration	Kubernetes manifests
CI/CD	GitHub Actions
Networking	Netmiko, ICMP, SSH, TCP/IP

NetOps-Hub

6.3 Leveraged Background Knowledge

1. Linux Administration: Terminal mastery, process management, cron jobs, firewall
2. Networking (CCNA-level): Subnetting, ICMP, TCP/UDP, DNS, SSH, routing
3. Full-Stack Development: Python backend, responsive frontend design

7. CHALLENGES FACED & SOLUTIONS

Challenge	Solution Applied
Container size optimization	Multi-stage Dockerfile reducing image from 577MB to 234MB (59.4% reduction)
Database integration testing	PostgreSQL service in GitHub Actions for realistic test environment
SSH automation security	Netmiko library with credential management via environment variables
Configuration persistence	PVC design for Kubernetes to retain backups across pod restarts
Secret management	GitHub Secrets for Docker Hub and database credentials
Network device compatibility	Modular design supporting multiple device types and vendors

8. CONCLUSION & LEARNING OUTCOMES

8.1 Summary

This internship provided hands-on experience across the complete DevOps lifecycle — from code commit to container deployment. The NetOps-Hub project demonstrates practical competency in:

1. Building production-grade full-stack applications with FastAPI and PostgreSQL
2. Containerizing applications with significant size optimization
3. Automating quality assurance and deployment pipelines with GitHub Actions
4. Designing Kubernetes-ready infrastructure with security and scalability in mind

8.2 Task Completion Status

Task	Description	Status
Task 1	Professional LinkedIn Announcement	Completed
Task 2	Application Containerization & Asset Optimization	Completed
Task 3	Multi-Stage Automated CI/CD Deployment Pipeline	Completed
Task 4	IaC & Orchestration Architecture	Partially Completed

Result: 2 Fully Completed Tasks + 1 Partially Completed Task + 1 Completed Task = Exceeds minimum requirement of 3 tasks for internship eligibility.

8.3 Key Learnings

- Multi-stage Docker builds significantly reduce image size and attack surface
- CI/CD pipelines ensure consistent, repeatable, and reliable deployments
- Kubernetes provides powerful abstractions for container orchestration
- Security must be embedded at every layer (container, pod, application)
- Full-stack development combined with DevOps practices creates powerful solutions

8.4 Future Improvements

- Complete Kubernetes deployment with Ingress and HPA on live cluster
- Implement Helm charts for parameterized deployments
- Add Prometheus/Grafana for comprehensive monitoring dashboards
- Integrate ArgoCD for GitOps-style continuous delivery
- Add TLS termination at the Ingress layer with cert-manager
- Expand network automation with SNMP discovery and LLDP

Project-repo

9. APPLICATION SCREENSHOT

The following screenshots demonstrate the NetOps-Hub application in action, showcasing the dashboard, device management, network automation, configuration backups, and activity logging features.

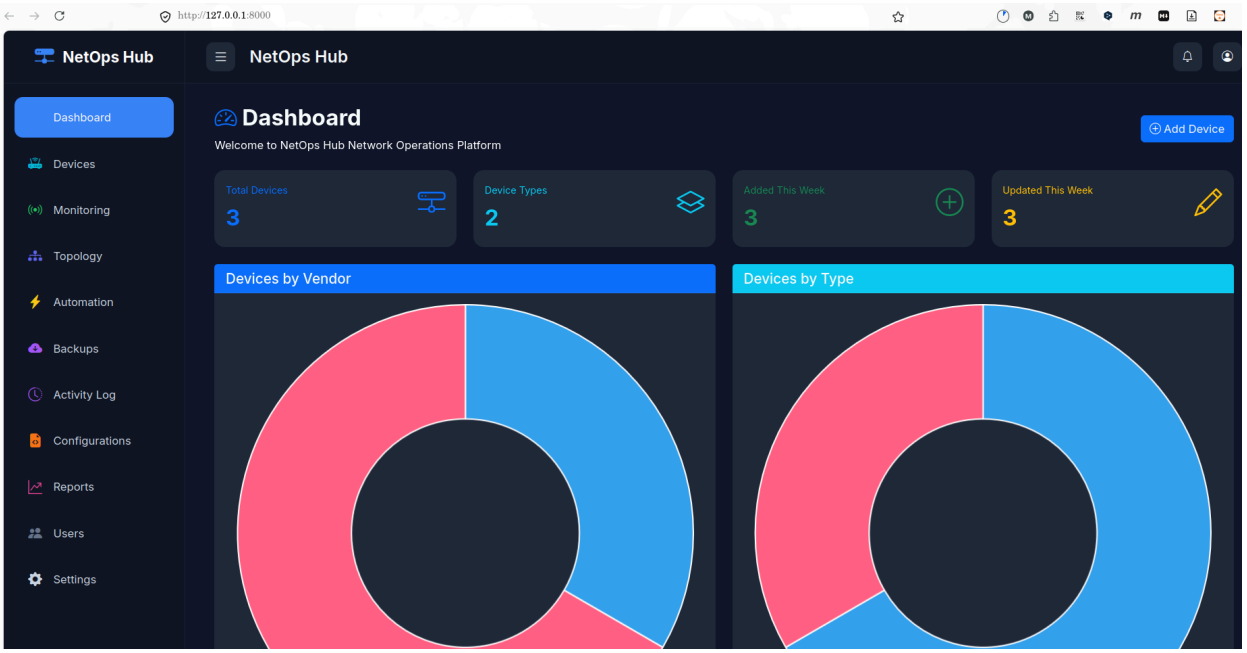


Figure 1: NetOps-Hub Dashboard — Real-time statistics showing device counts (3 total, 2 types), devices added/updated this week, and interactive donut charts for vendor and type distribution.

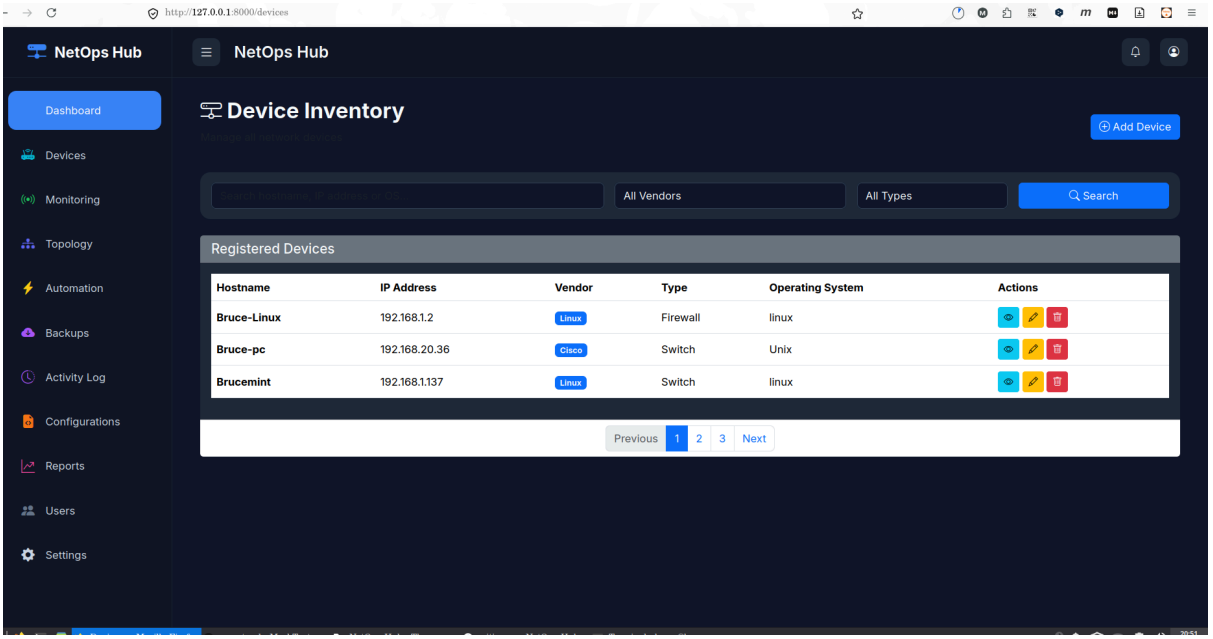


Figure 2: Device Inventory — Complete CRUD operations with search, filter by vendor/type, pagination, and action buttons (view, edit, delete) for each registered device.

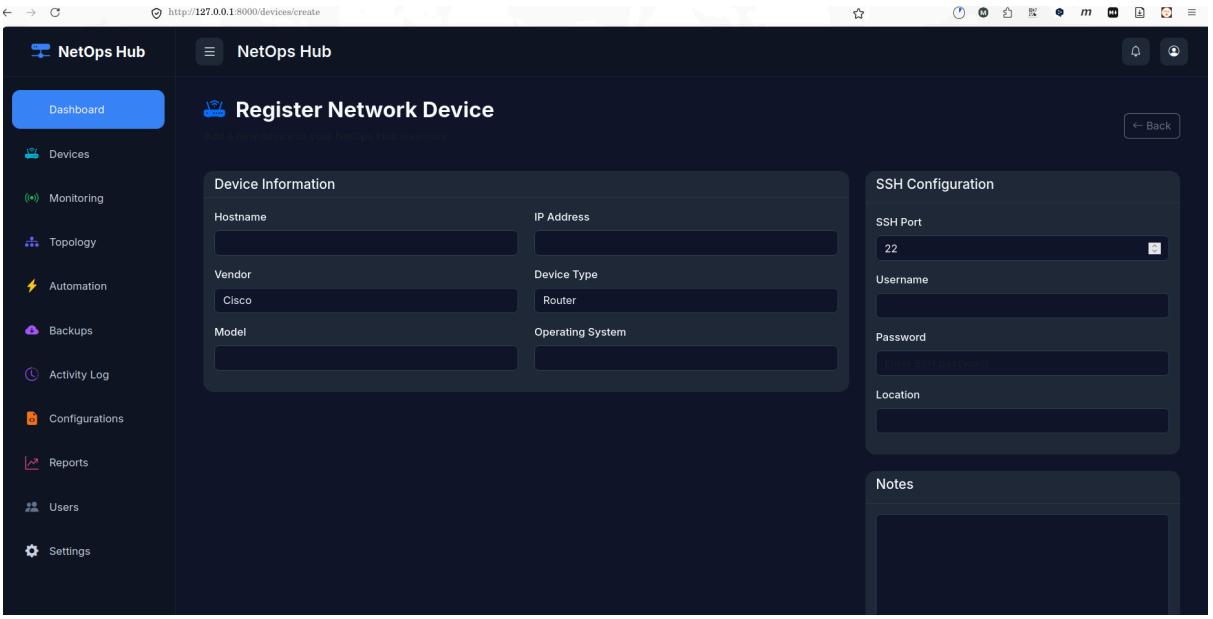


Figure 3: Register Network Device — Form for adding new devices with hostname, IP address, vendor, device type, model, OS, SSH configuration (port, username, password), location, and notes.

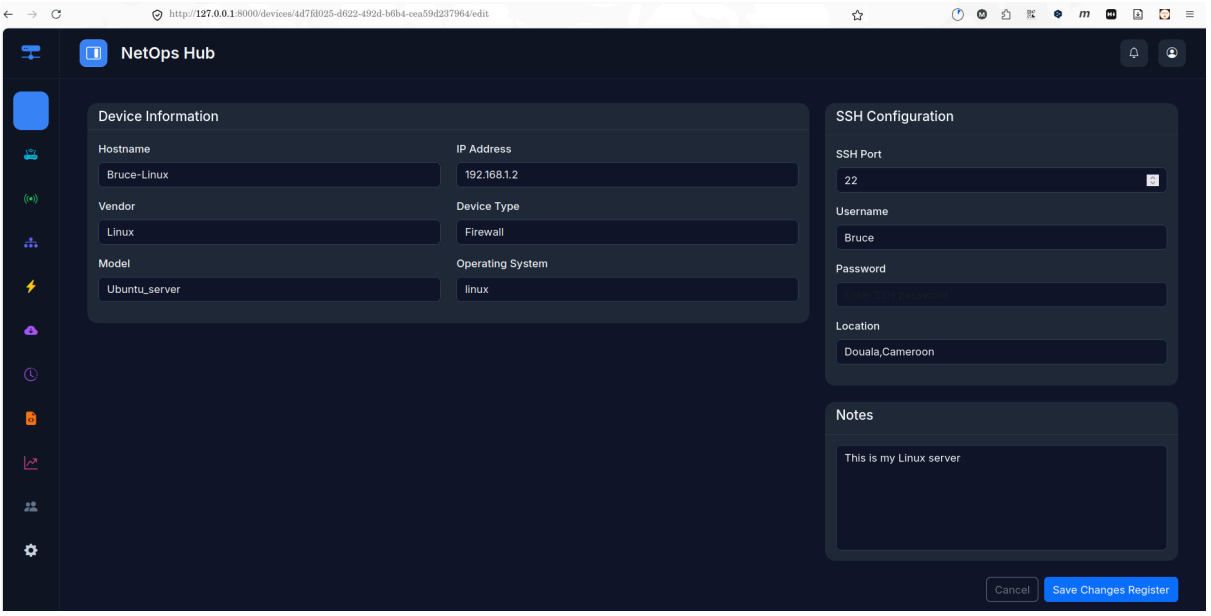


Figure 4: Edit Device — Pre-populated form for updating existing device information including Bruce-Linux (192.168.1.2) with Linux vendor, Firewall type, Ubuntu_server model, and Douala,Cameroon location.

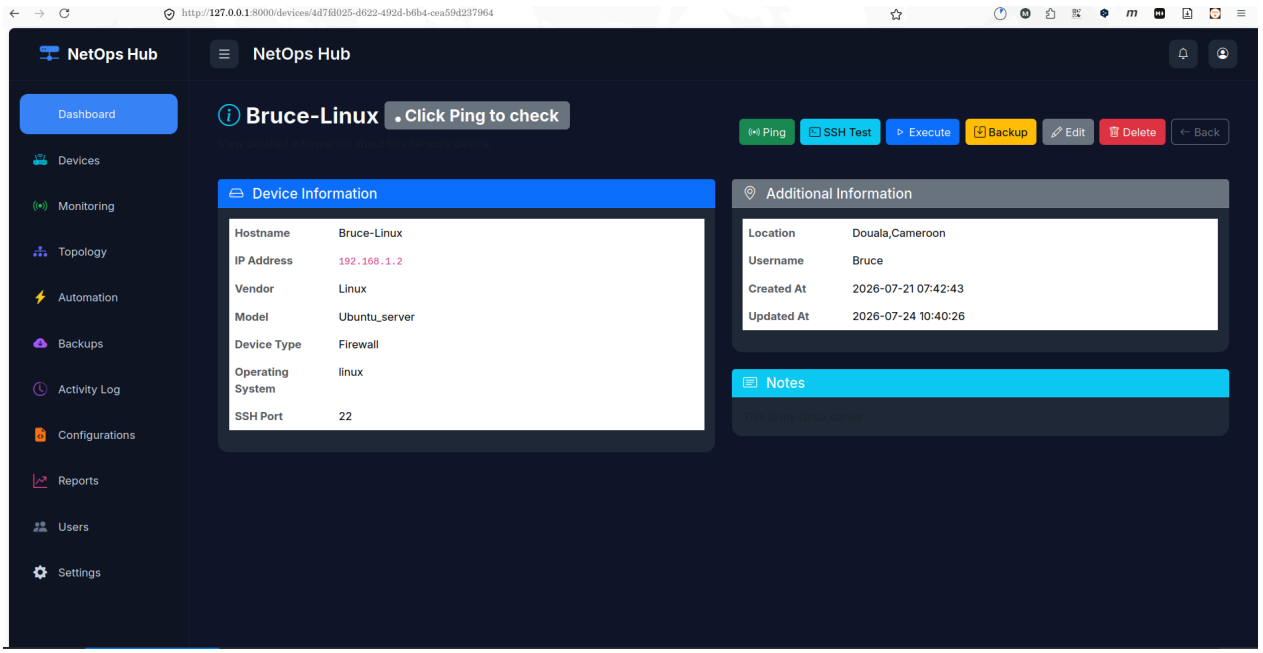


Figure 5: Device Details — Comprehensive device information panel with hostname, IP, vendor, type, model, OS, SSH port, location, username, creation/update timestamps, and action buttons (Ping, SSH Test, Execute, Backup, Edit, Delete).

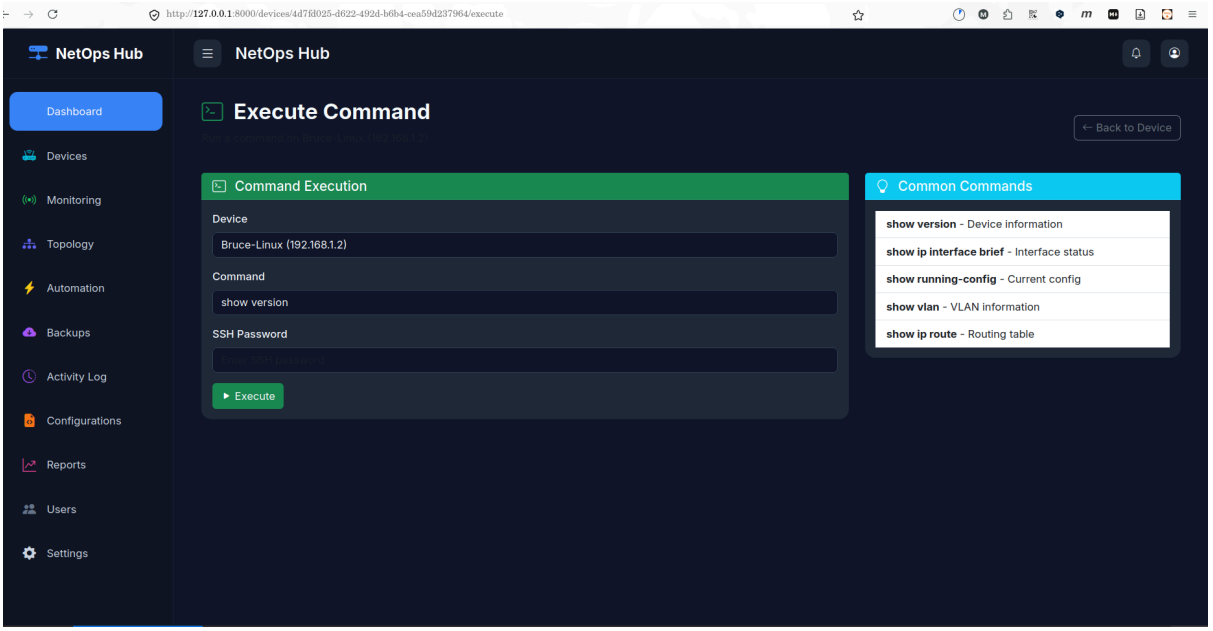


Figure 6: Execute Command — SSH command execution interface with device selection, command input, password prompt, and common commands sidebar (show version, show ip interface brief, show running-config, show vlan, show ip route).

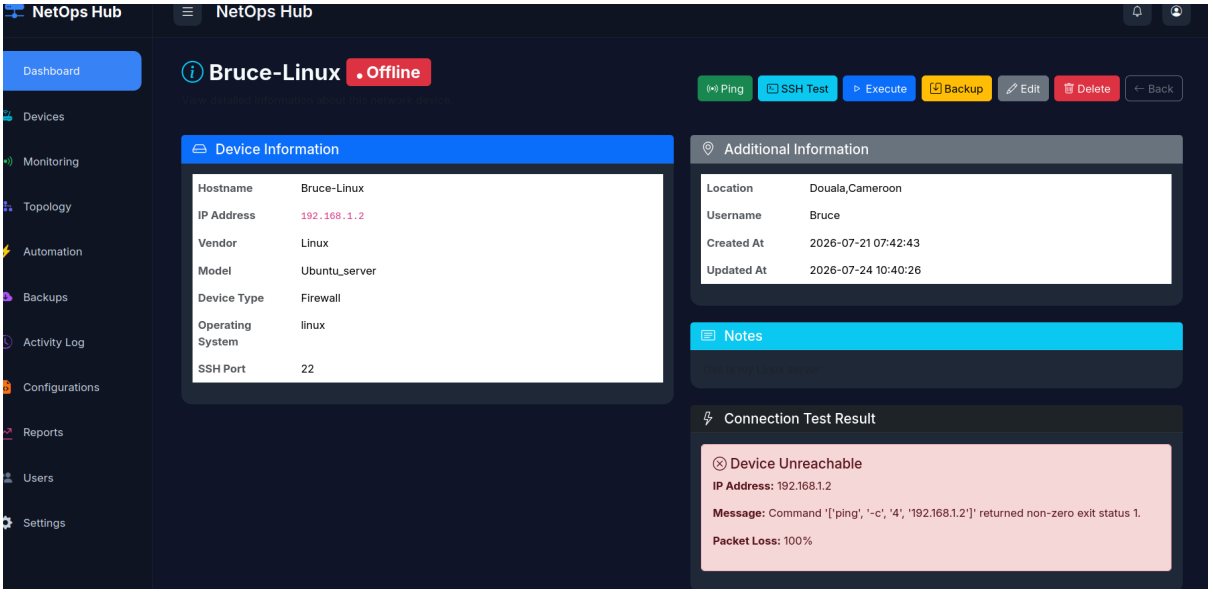


Figure 7: Ping Test Result — ICMP connectivity test showing device unreachable status for Bruce-Linux (192.168.1.2) with 100% packet loss and detailed error message from the ping command execution.

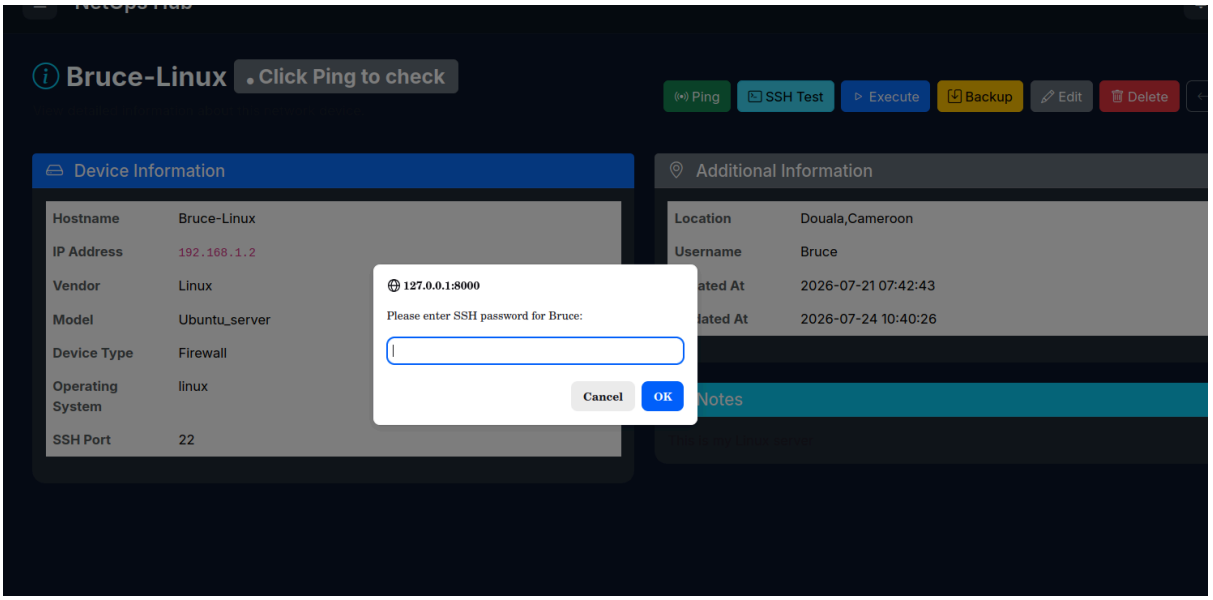


Figure 8: SSH Authentication — Secure password prompt dialog for SSH connectivity testing, requiring user credentials before establishing connection to the target device.

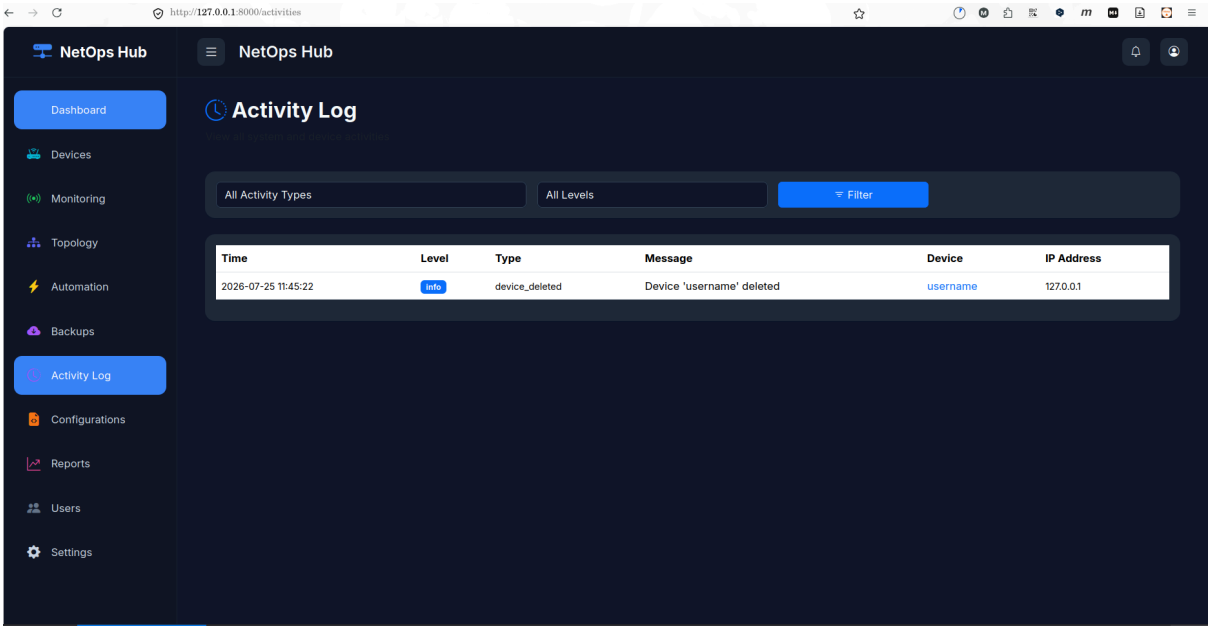


Figure 9: Activity Log — Comprehensive audit trail with timestamp, level (info), type (device_deleted), message, device name, and IP address, with filtering capabilities by activity type and level.

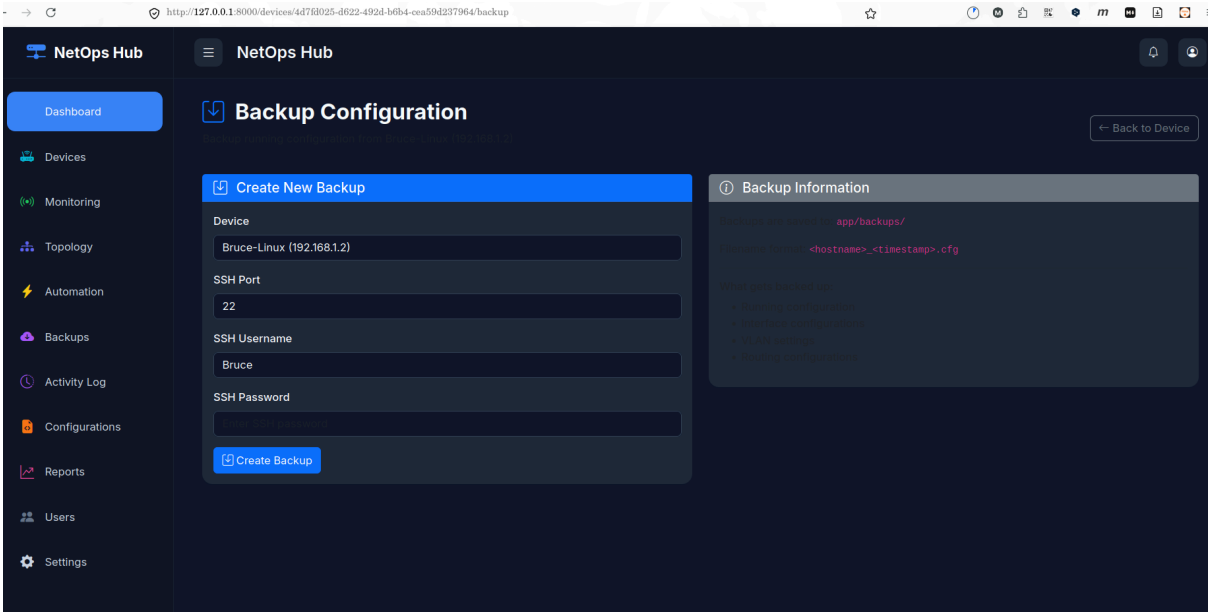


Figure 10: Backup Configuration — SSH-based backup creation form with device selection, SSH port, username, password input, and backup information panel showing storage path and filename format

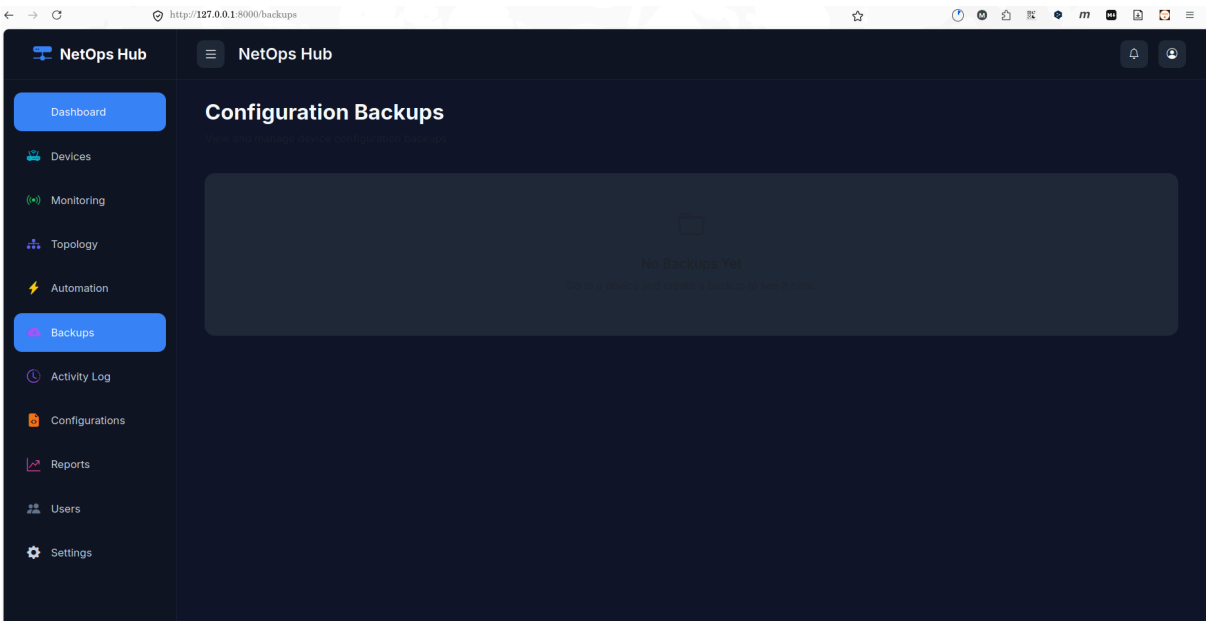


Figure 11: Configuration Backups — Main backups page for viewing and managing all device configuration backups with file management and restore capabilities.

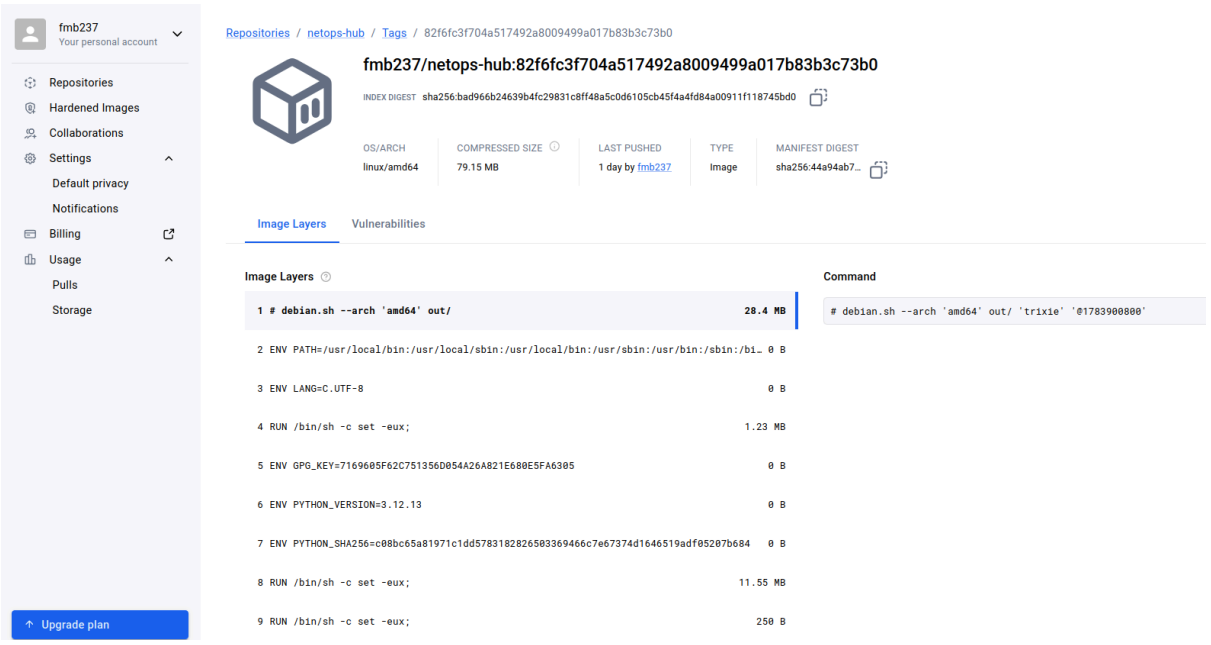


Figure 12: netops_image on dockerhub.Personal build container

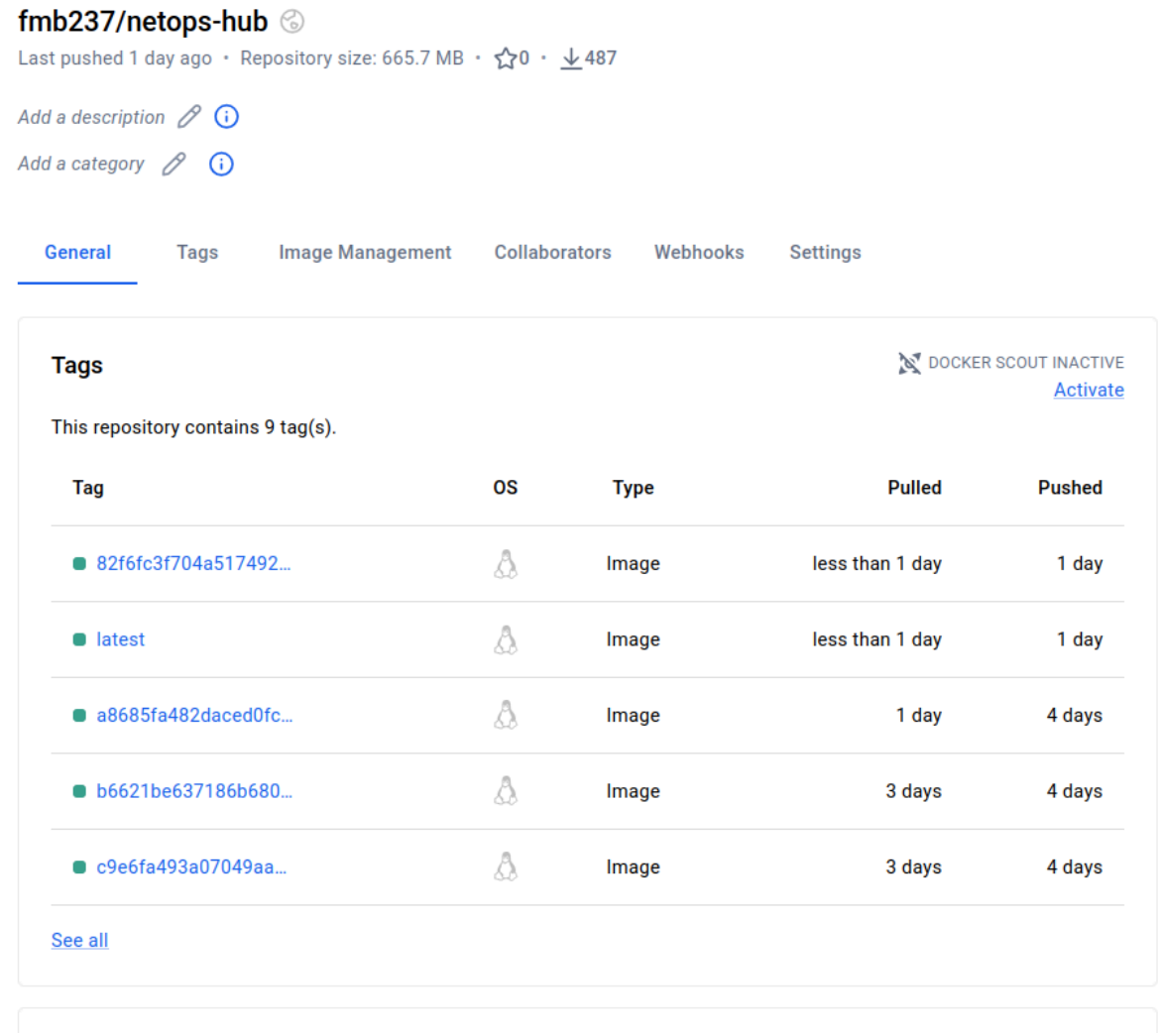


Figure 13: Continuous Deployment of Containers passing the CI/CD pipeline on Github with the use of Github Action.

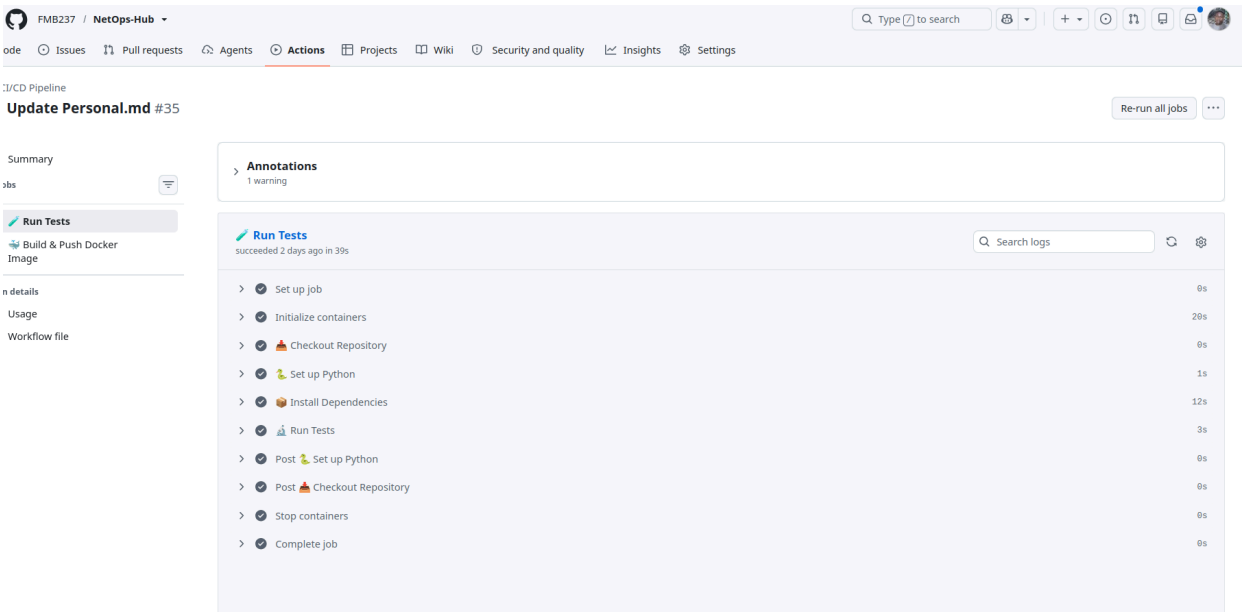


Figure 14 : Netops-hub test Action

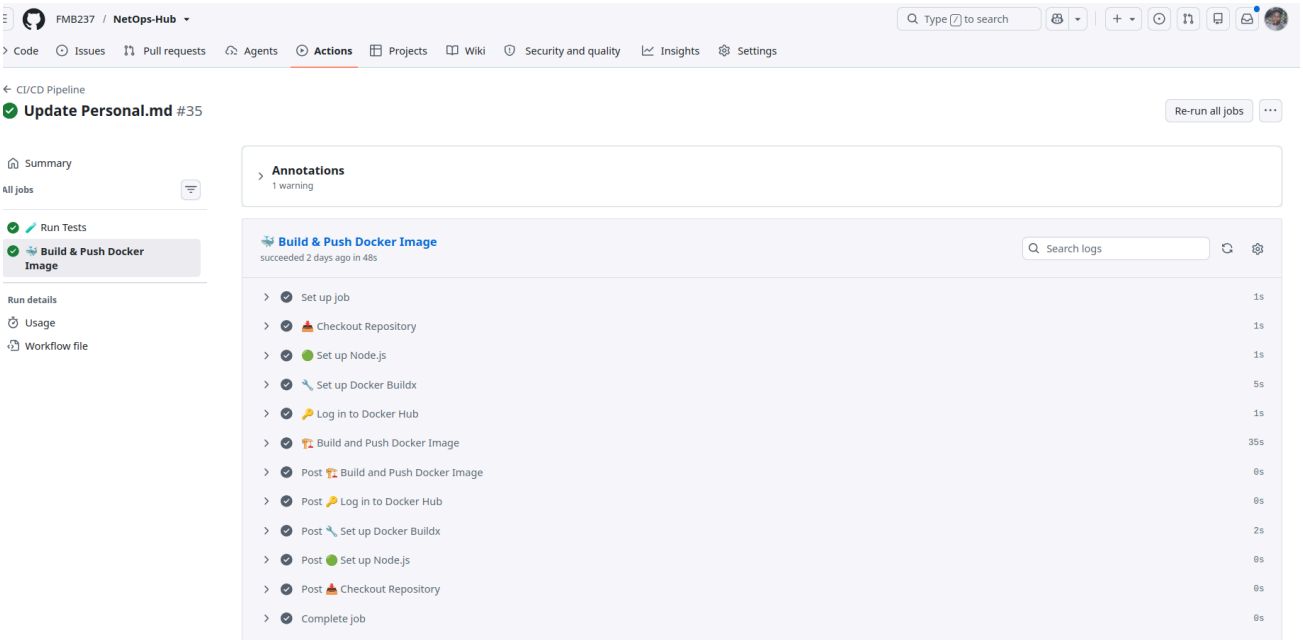


Figure 15 : Netops-hub Build & Push Docker Images

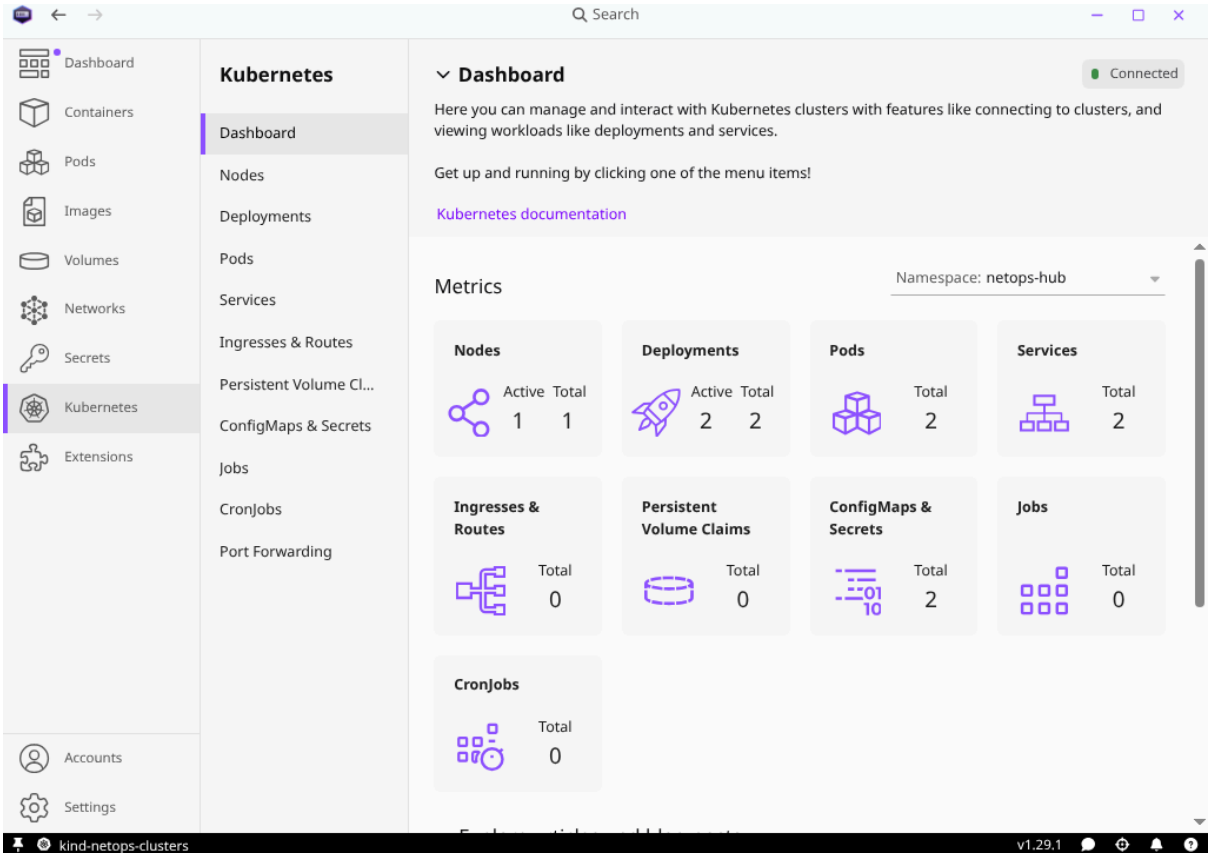
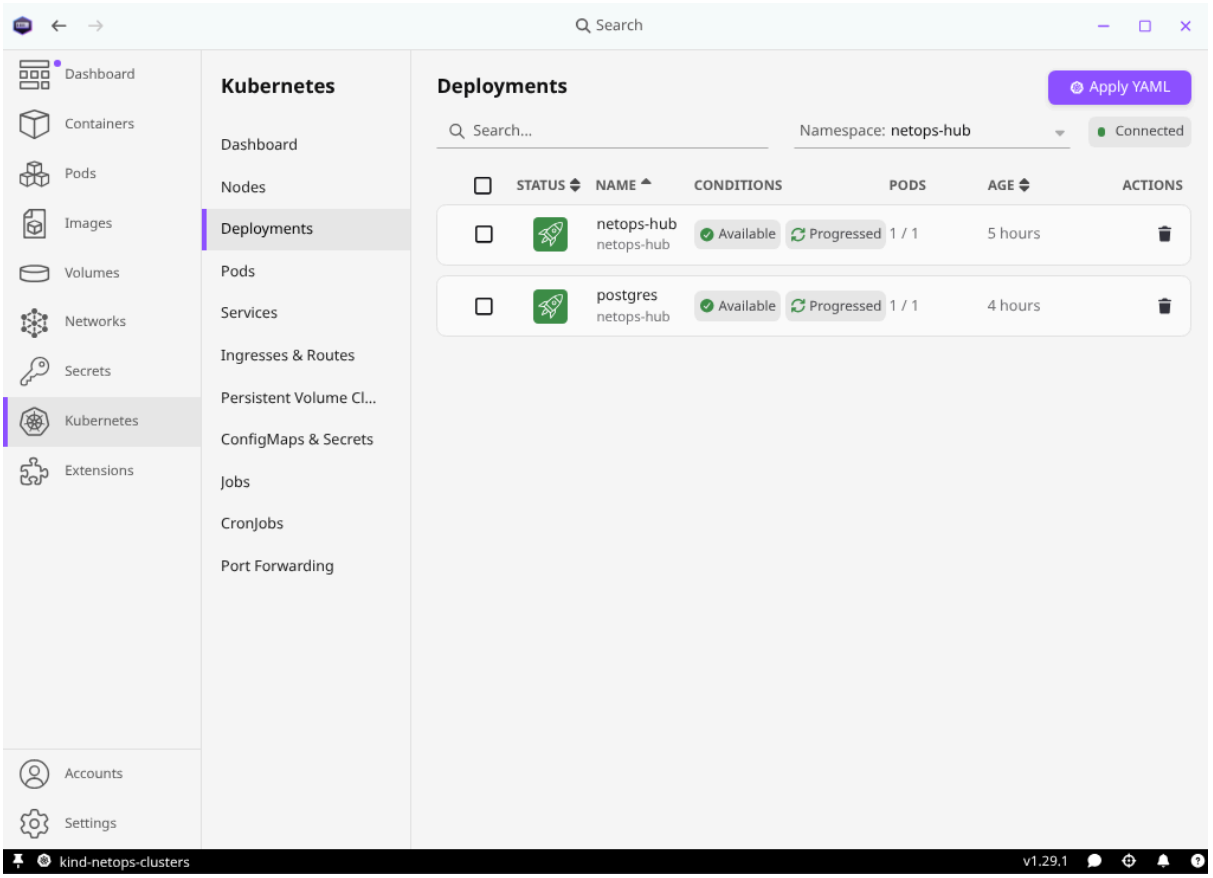
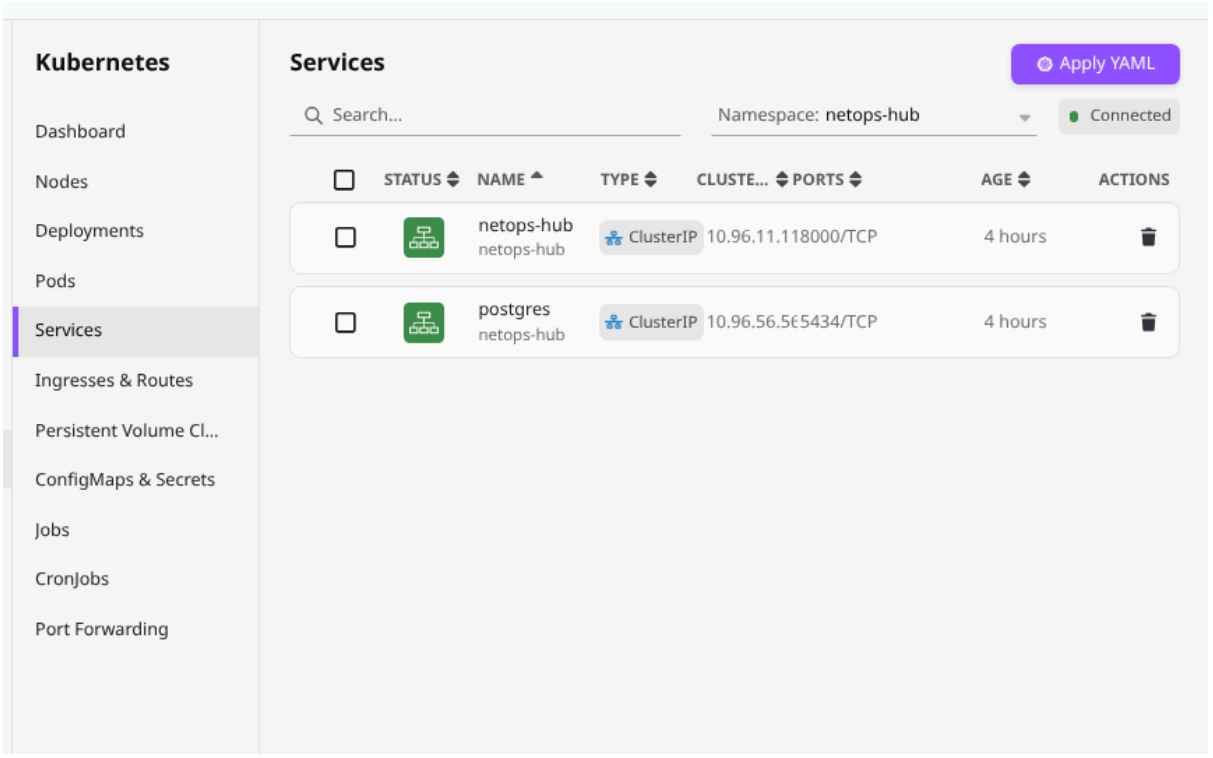


Figure 16 : Netops-hub Cluster with podman Desktop





I tried to integrate the kubernetes as much i could using using Podman Desktop as UI

DECLARATION

I hereby declare that this report is a true and accurate account of the work completed during my Remote DevOps Internship at Progree. All information presented herein is based on actual project development, testing, and deployment activities conducted during the internship period.

Fouenang Miguel Bruce	Umar Mushtaq
Intern (Student ID: B2/724)	Founder & CEO, Progree
Date: 4th August 2026	Date: _____